

CONCEPTOS — SESIÓN 4

Estructuras de Control: Repetición (MIENTRAS y REPETIR)

Ciclos, acumuladores, validación de entrada

¿Por qué necesitamos ciclos (repetición)?

Imagina que necesitas escribir los números del 1 al 1000. ¿Escribirías 1000 líneas de código? Eso es absurdo. Un CICLO es una estructura que REPITE un bloque de código mientras se cumpla una condición. En lugar de 1000 líneas, escribes 3 líneas que repiten automáticamente.

La estructura MIENTRAS (WHILE)

Mientras (condición) Hacer [código que se repite] FinMientras

El funcionamiento es así: la computadora VERIFICA la condición ANTES de entrar. Si es Verdadera, entra y ejecuta el código. Luego vuelve a verificar. Si sigue siendo Verdadera, repite. Cuando la condición es Falsa, SALTA el código y continúa después del FinMientras.

CRÍTICO: si la condición es Falsa desde el inicio, NUNCA entra al ciclo. El código dentro puede nunca ejecutarse.

Ejemplo: Mientras (contador <= 5) Hacer. Si contador empieza en 10, la condición 10 <= 5 es Falsa, el ciclo NO ejecuta.

La estructura REPETIR-HASTA (DO-WHILE)

Repetir [código que se repite] Hasta Que (condición)

La diferencia CRUCIAL: la computadora VERIFICA la condición DESPUÉS. Esto significa que el código se ejecuta SIEMPRE AL MENOS UNA VEZ, incluso si la condición es Falsa desde el principio.

Nota sobre la gramática: "Hasta Que" significa "hasta que se haga verdadero". Es lo opuesto a "Mientras". Mientras dice "repite MIENTRAS sea verdadero". Hasta Que dice "repite HASTA QUE sea verdadero".

Diferencias: MIENTRAS vs REPETIR

MIENTRAS verifica la condición ANTES de entrar. Si es Falsa, NO entra. Es para casos donde tal vez no necesites repetir.

REPETIR verifica la condición DESPUÉS de ejecutar. Siempre entra al menos una vez. Es para menús, validación de entrada, o cuando garantizas que NECESITAS al menos una ejecución.

Elementos clave de un ciclo

Variable de control: Es la variable que CAMBIA en cada iteración del ciclo. Ejemplo: $i < i + 1$. DEBE cambiar obligatoriamente, o el ciclo se vuelve infinito (repite para siempre).

Acumulador: Es una variable que SUMA valores a través de las iteraciones. Siempre comienza en 0. En cada iteración, se suma un nuevo valor: $\text{suma} <- \text{suma} + \text{numero}$. Si suma era 10 y numero es 5, suma AHORA es 15.

Bandera (flag): Es una variable Lógica (Verdadero/Falso) que controla la ejecución del ciclo o indica si algo fue encontrado. Ejemplo: $\text{encontrado} <- \text{Falso}$. Cuando se cumple una condición, $\text{encontrado} <- \text{Verdadero}$.

Validación de entrada con REPETIR

Uno de los usos más prácticos de REPETIR es VALIDAR que el usuario ingrese datos correctos. Por ejemplo, pedir una edad entre 0 y 150.

Repetir Escribir "Ingrese su edad (0-150):" Leer edad Si (edad < 0 O edad > 150) Entonces Escribir "Inválido. Intente nuevamente." FinSi Hasta Que (edad >= 0 Y edad <= 150)

El código se repite HASTA QUE edad esté en el rango correcto. El usuario NO PUEDE salir del ciclo sin ingresar un dato válido. Esto se llama VALIDACIÓN: verificar que los datos sean correctos antes de usarlos.

Ciclos infinitos: Cómo evitarlos

Un ciclo INFINITO ocurre cuando la condición NUNCA se vuelve Falsa. El programa se queda repitiendo para siempre y se "congela".

Causa principal: Olvidar cambiar la variable de control. Ejemplo:

```
contador <- 1 Mientras (contador <= 5) Hacer  Escribir contador  " AQUÍ FALTA: contador <- contador + 1"  
FinMientras
```

Sin la línea que incrementa contador, siempre será 1, $1 \leq 5$ siempre es Verdadero, INFINITO.

Para evitar ciclos infinitos: SIEMPRE verifica que la variable de control CAMBIE dentro del ciclo.